ICT2113 SOFTWARE REQRS:
NOW & NEAR FUTURE

SINGAPORE INSTITUTE OF TECHNOLOGY

12. UML: Giant No More – Less Relevant w/Lightweight

Prof Kevin

“Beneath this mask there is an idea … and ideas are bulletproof.”

V for Vendetta, 2005

Idea of UML: bring **order** to **extreme chaos** in **modeling** for sw dev. From flowcharts (1960s) to structured systems (‘80s) to object-oriented & lightweight (‘90s), it was highly likely that every new proj required its developers to learn new modeling language. In starker terms, developers were lucky in 10 sw dev projs if they could **reuse modeling** scheme **once**. Initially, UML was wild success. Sw community loved it, and customers were happy with improved documentation. But it had **Achilles’ heel**; it’s strictly heavyweight. Lightweight needs little of it … why won’t you die?

Agenda: Giant no more in lightweight

1. Previous week's learning. (Do self-review LS#4 just to reconnect with what modelling is, and models in RCM.)
2. Today's learning outcomes.
3. Video – "UML and why you need it".
4. Today's new learning material.
5. Active learning – three text-to-UML-diagram.
6. Epilogue.

Previous week's learning

Six slides

Remember why review of previous week's learning important!

Strengthen memory retention.

Highlight knowledge gaps.

Prepare for acquiring new material (per iterative learning approach in HE institutions).

Overview of UseCase2.0 methodology

In 2011 eBook “Use-Case 2.0: The Guide to Succeeding with Use Cases”, Jacobson announced his new way of applying UseCases that is lightweight, lean, scalable, versatile, easy-to-use. Back in 1996, Kent Beck (XP lead-creator) says he was inspired by Jacobson’s UseCases — jump 15 years later, Jacobson says with UC2.0, “inspiration has flown in opposite direction”.

UC2.0 is disruptive technology to traditional Agile development that uses User stories. Such projects often suffer from fragmentation debt — where over-all system context, i.e., big picture, is lost in sea of disjointed PB items. UC2.0 is beyond-worthy alternative by handling all scales of project, large to small scale.

It achieves this through its unique three-method approach. First, it preserves reliable UseCase model to provide big picture map of new sw. Second, it re-works “stories of usage” to ensure more robust specification that anticipates complexity before it hits development phase. Third, it introduces Slices to ensure that every realization delivers functional piece of new sw that is both architecturally sound and immediately valuable to user.

Summary of three methods in UC2.0

parent-UC

- Specification of some part of biz ops as sequences of related (GPV) & dialogic transactions with user(s)
- Written in two mediums: graphic & matrix.
- Stable anchor to manage fluid implementation.
- Weight-neutral: handles both heavy- & lightweight dev proj.
- Effectively written: graph by OMG's UML, and, matrix by Alistair's 2000-1 book, "Writing Effective Use Cases".

Story

- Short description of dialogical transactions between user & system, i.e., "story of way of usage" of UC.
- Authored by user or BA.
- Three categories of usage: basic, alternate & exception
- UC comprised of several Stories that cover every way, default, normal, and flawed, that it is used.
- Effectively written via Alistair's casual + low ceremony template.

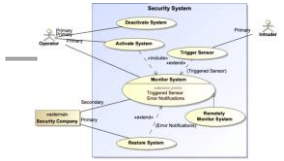
Slice

- Technical specification of small portion of UC's functionality that undergoes realization per lightweight norms.
- Fcn drawn from Stories, oriented vertically per BCE from Jacobson's Analysis Model (1992), and specified into end-to-end segment through platform's sw architecture.
- Effectively written via academic-purposed makeshift template that follows Jacobson's guidelines.

UseCase2.0 process model

SINGAPORE INSTITUTE OF TECHNOLOGY

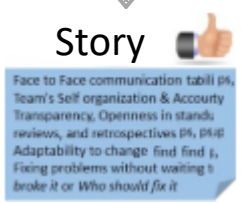
Users **prioritize** each UC for MVP (2)



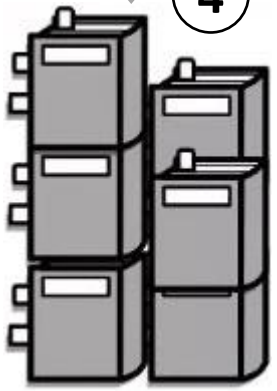
(1) Users draw **Use-Case model** of new sw, and list **Auxiliary details**

Users write **Stories** for **priority** UC (3)

Story



(4)

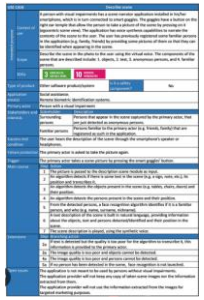


Store Stories in **PB** per UC & priority

Users

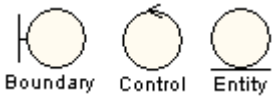
Developers

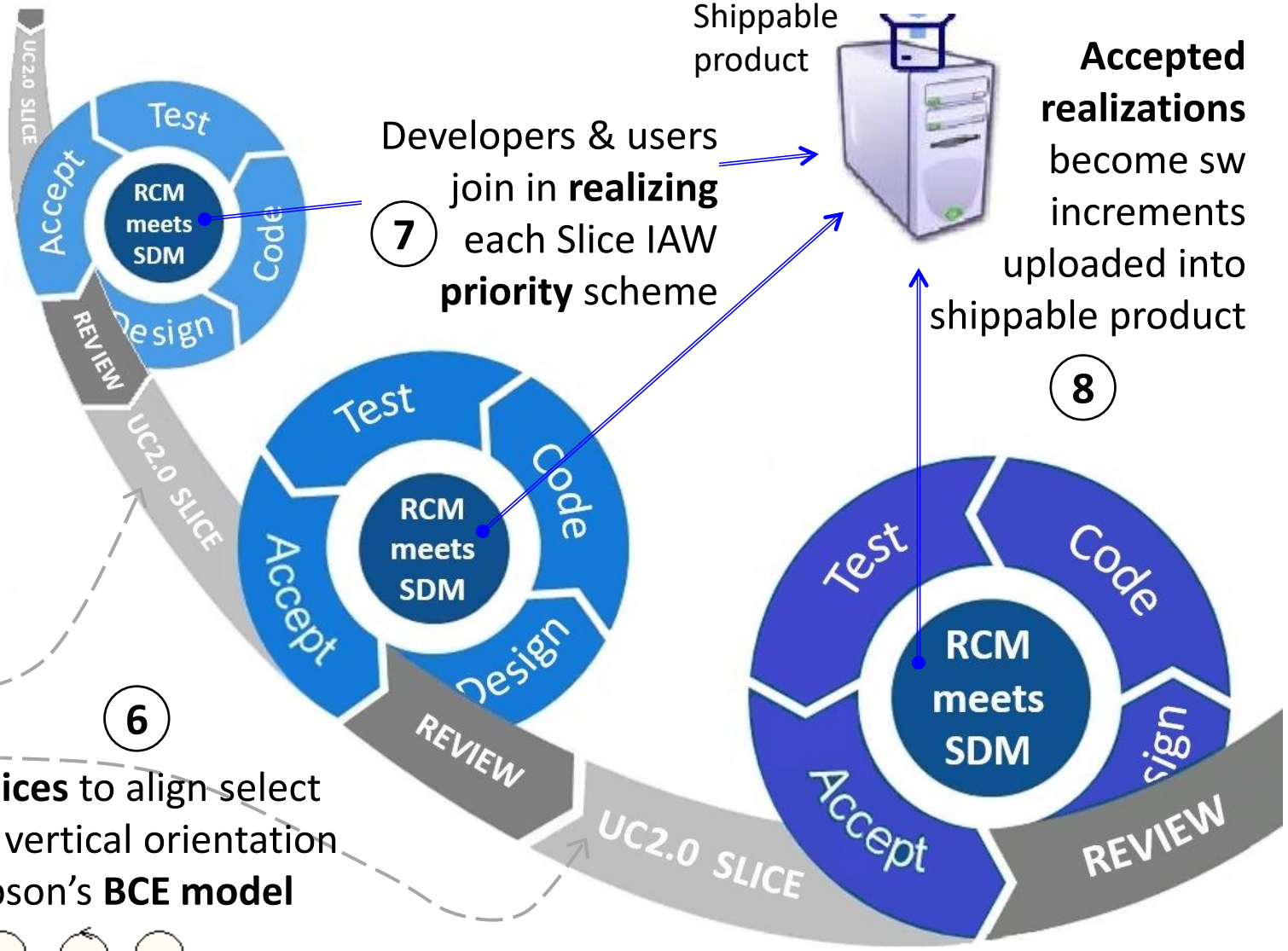
Developers join users to **specify** full reqrs from Stories (5)



Slices

(6) Write **Slices** to align select reqrs in vertical orientation of Jacobson's **BCE model**





(7) Developers & users join in **realizing** each Slice IAW **priority** scheme

Shippable product

Accepted realizations become sw increments uploaded into shippable product (8)

Auxiliary details

For: < <"Model of new sw" <name>> or
< "UseCase" <name>> or
<"Story" <name> > >.

Effective date:

List of details:

1. **Assumptions**:
2. **Business rules**:
3. **Constraints**:
4. **Pre- & post-conditions**:
5. **Technical details**:
   a. **Database elements & parameters**:
   b. **Load & stress parameters**:
   c. **Performance benchmarks**:
   d. **Platform & IT compatibility**:
   e. **Quality features**:
   f. **UI & UX**:

Makeshift template (see above). Content for each field is free-text; if nil, say "none". Add or delete technical items as applicable.

UC-Slice prioritization scheme

Prioritization conducted per one of two schemes: minimum viable product (MVP), and Walking skeleton. Their common purpose is to minimizes risk and validate reqrs & assumptions early to avoid painful rewrites later. Here are their unique aspects:

- **MVP**. Coined by Frank Robinson at SyncDev in 2001, and later popularized in 2011 "The Lean Startup". Has enough features for early adopters to meaningfully use and provide feedback on what may need adjustment.
- **Walking skeleton**. Coined by Alistair Cockburn in 2004 "Crystal Clear: A Human-Powered Methodology for Small Teams". Technical proof of concept of architecture with minimal features; backbone of development.

Walking skeleton is realized first before MVP, ensuring that decisions pertaining to biz ops can at least have foundation of new product's technical viability. Prioritization is not static; it can be redirected as more increments of shippable product are released.

Alistair's effective writing of Stories

Alistair Cockburn, independent consultant, and co-signatory on 2001 Agile Manifesto, started his decades-long association with writing UseCases when he visited C3 project in 1998, expecting UseCases but getting User stories (see LS#10). In 2000/2001, he published his tome "Writing Effective Use Cases", cementing his eminence in writing UseCases. (Interestingly similar to way Mike Cohn gained his estimable status in effective writing of User Stories.)

UseCase templates in his 2000/2001 book are elaborations of varying complexity of matrix models. As such, they are not appropriate for lightweight development, except for one of them, i.e., 'Casual' & 'low-ceremony' (see below). Essentially, this is most useful to UC2.0 for effective writing of Stories.

Use Case 1: Buy something

The Requestor initiates a request and sends it to her or his Approver. The Approver checks that there is money in the budget, check the price of the goods, completes the request for submission, and sends it to the Buyer. The Buyer checks the contents of storage, finding best vendor for goods. Authorizer: validate approver's signature. Buyer: complete request for ordering, initiate PO with Vendor. Vendor: deliver goods to Receiving, get receipt for delivery (out of scope of system under design). Receiver: register delivery, send goods to Requestor. Requestor: mark request delivered.

At any time prior to receiving goods, Requestor can change or cancel the request. Canceling it removes it from any active processing. (delete from system?) Reducing the price leaves it intact in process. Raising the price sends it back to Approver.

Effective writing of Slice

Being technical specification involving database elements, Interface contracts & platform's sw architecture, Slice is written more formally and with more technical jargon than Stories. But, as with Stories, Slices have no accepted writing convention. Instead, it has only guidelines from its creator.

For academic purposes, makeshift template is appropriate. (See right) Said guidelines by Jacobson are converted into Slice Technical Specification (STS) template. Examples provided in next slide.

Title: Slice unique ID and name.

Context:

- **Effective date**: Date in Julian format as per policy.
- **Parent UC:** ID and name of parent UC.
- **Story list**: Prioritized **list** of the specific Stories used.
- **Prelim:** List of pre-conditions plus description of current situation to ready for realization according to its dictates.

Narrative: Derived telling of Stories sourced for vertical stack. Also include Auxiliary details specific to used Stories.

Scope:

- **Category:** basic / alternate / exception.
- **Priority**, **MVP**, **Status**, **KIV**, , **etc**.

Tech details: Details of actions, including changes, to specific Architectural components in vertical stack, including UI (Boundary), Logic (Control), and Database (Entity). Also API/Interface contracts between Layers, and from internal system to external. Also, infrastructure setup, DevOps and environmental requirements necessary to deploy and run Slice.

Acceptance: List of Acceptance criteria (AC) tests, including binary checks {Y, N}, to confirm the Slice's vertical stack is 'done/fully in place'.

Today's learning outcomes

One slide

Remember why knowing learning outcomes important!

Roadmap of learning goals & expectations.

Encourages developing preparation & study strategies.

Promote engaging in learning process.

Facilitate managing learning material & effort.

LO#12: Modelling giant UML's decline

After today's lesson, student will be able to explain …

- … six phases of sw dev proj modeling.
- … immediate benefits of UML to heavyweight developers.
- … UML's semantic backplane.
- … four reasons for UML's decline in usage
- … four selective & pragmatic uses currently of UML by developers.

Draw four each of vertex & edge shapes for structure & behaviour MEs that represent notational self-consistency.

These are objectives I plan for you to achieve in this session.

Your objectives should sync with these, but don't need to be verbatim identical.

Video

One slide & 4:10 mins clip

Things to keep in mind before viewing

Today's business challenges are daunting, especially with escalation of emerging technology and consequent new ways and means of working. Yet, in all this turmoil, sw industry finally has solution to disarray in designing, communicating, and documenting its products and services under development: Unified Modeling Language (UML).

You may have seen two videos circulating on YouTube, "I hate UML" and "What went wrong with UML tools? (and why do we need another?)" but usage statistics are okay, and that UML is industry staple, albeit in decline.

Instead, I'm showing you pro-UML video by Károly (he goes by Karl) Nyisztor @knyisztor, long time contributor of SWE videos in his channel. He provides sober and compelling arguments about why we should embrace UML. As someone whose spend decade on learning all of UML secrets, I heartily support this video. I hope you will too. Signal when you understand, and are ready to begin.

Anything you are uncertain about? …

Ask, and let me help clarify

Take quick break

300 sec countdown – ends when fully filled

Let's continue with LS

Today's new learning material

Nine slides

Sw dev proj modeling phases (cont)

When cables & switches gave way to stored programs in 1949, modern sw was born, but not without major problem: how to direct coders to build right code right? Conveying instructions verbally proved to be highly ineffective because of, difficulty of abstraction, imperfect repetition, and diminishing returns on time taken. Modeling was (and still is) answer, but continual growth in sw platforms and proficiency obliged modeling to mature similarly in capability and usability. Since World War II to today, modeling has gone through five major phases, and now on its sixth.

Phase 1 Flowcharting, 1947 to end-1960. It started with von Neumann and Goldstine proposing that mathematicians design and write algorithms for programs, that could be recorded into flow diagrams (later flowcharts), and then transformed by coders into specific machine instructions. This quickly took hold of industry, was standardized internationally, and even became part of programmer's training. But more complicated machines and operating systems proved to be flowcharts' undoing decade later.

Phase 2 Developer-centric flow models, 1970 to 1980. These new diagrams were born inside heavyweight SDMs, and proprietary to, developer organizations like IBM and independent pioneering consulting firms. Some clung onto flowcharting, but repurposed it to show what program does rather than how programmer should build it. At least 12 major sw dev proj modeling conventions took centre-stage during 1970s, including Structured Design by Yourdon & Constantine (1975) and System Analysis & Spec by Tom DeMarco (1978).

Phase 3 SSADM, 1980 to 1990. Conceived by UK government telecommunications agency, this massive methodology with its regime of software diagrams dominates development of heavyweight information systems up to today. It combined new methods with appropriated leading SDMs of past decade, including Checkland, Constantine, DeMarco, Jackson, and Yourdon; its major models are, data & control flow diagram, entity-relationship diagram, state transition diagram, structure chart, decision tree & table, and data dictionary.

Sw dev proj modeling phases (cont)

Phase 4 Proprietary structured & OOP modeling, 1990 to 1996. Several of earlier heavyweight SDMs converted to object-oriented (OO), e.g., Yourdon & Coad method in 1988. Plus, there appeared dozens more new consulting firms' SDMs, some heavyweight and others lightweight, that were OO from their inception, and bore their own proprietary modeling techniques. But, these divergent SWE practices resulted in a) professional schisms, b) notation creep that deteriorates active understanding, and c) diagram archives that disintegrated as superceded notation could not be maintained. Easily 20 different model schemes.

Phase 5 UML, 1996 to 2007. This was essential resolution for burgeoning chaos. It is completely independent of any SDM or industry, so it makes modeling conform to industry, rather than other way around. Importantly, it integrates its semantic backplane and notational self-consistency with OMG's international standards for model-driven architecture, meta-object facility, and XMI model interchange. Yet, its heavy reliance on extensive documentation contributed to its decline once Agile movement took hold.

Phase 6 Pragmatic & text-based modeling, 2008 to present. With the rise of Agile development and Continuous Integration/Continuous Deployment (CI/CD) pipelines, lightweight, text-based modeling tools like PlantUML and Mermaid have become prominent. This allows diagrams to be version-controlled with code and automatically updated, solving documentation decay problem of previous phases. Also, it complements Domain-Driven Design (DDD) methodologies and informal collaborative techniques like Event Storming, with its just-in-time (JIT) communication rather than exhaustive, upfront documentation.

Initial reactions to UML: Positive, later muted

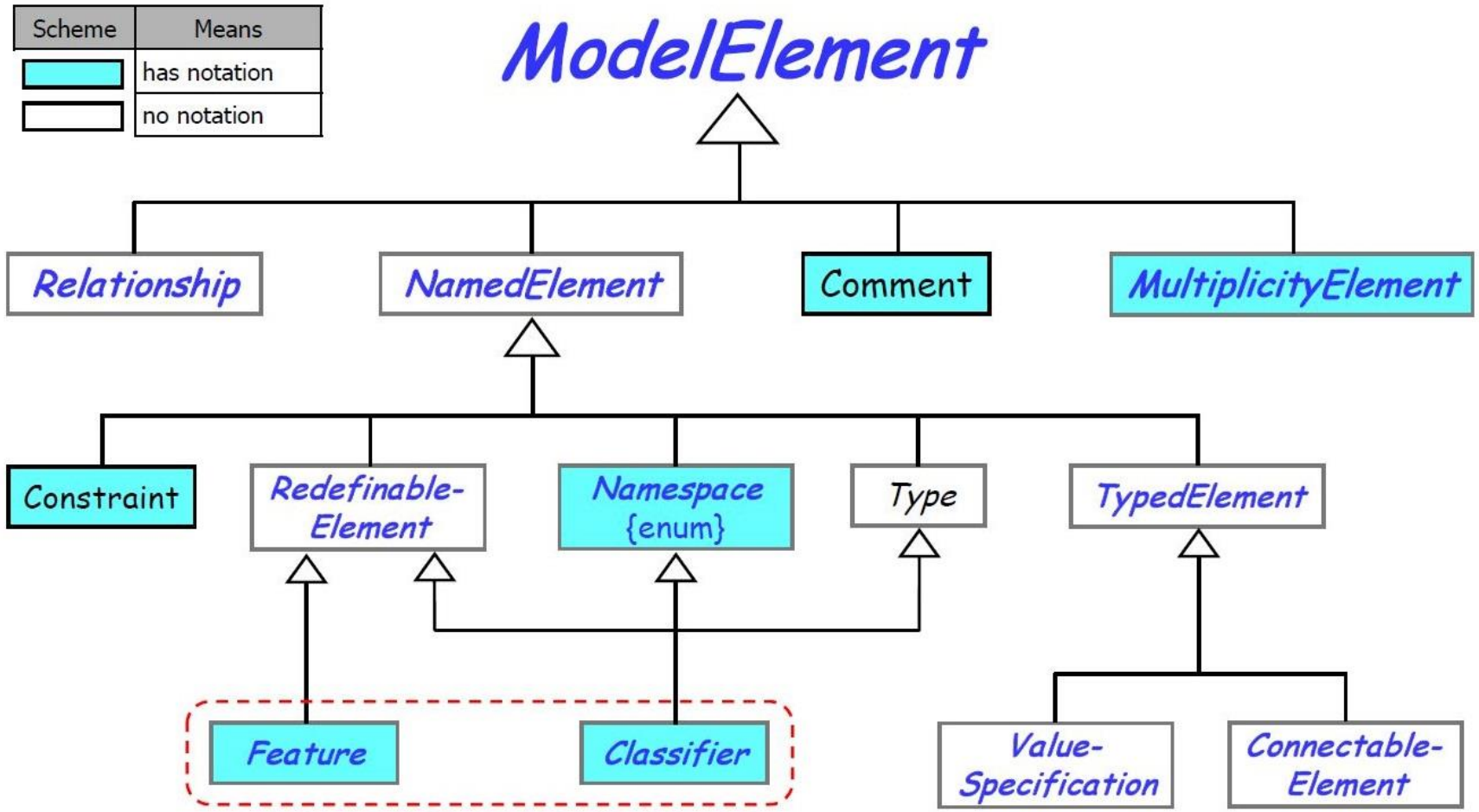
By heavyweight proj developers. Approximately 71 to 85% of early practitioners reported using UML Class diagrams for functional and data structure modeling. Immediate benefits were:

- **Code generation from model (Eclipse)**. Integration of UML with MOF standard supported use of CASE tools that could generate significant portions of code automatically from Class diagram, thereby increasing productivity and ensuring consistency. (However, tool support for this didn't go much farther.)
- **Improved communication**. With its open and pedantic standard, UML bridges communication gap among diverse stakeholders & developers.
- **Iteratively do designing**. UML's range of formedness of models, allows for developers to design big-picture, then design inside, and so on. Complex systems could be abstracted and then drilled-down, making all models interoperable, regardless of abstraction.
- **Preserving archives**. There are numerous large IT companies providing UML's CASE tool for diagram editing. Hence, companies can preserve their archived models so long as those IT companies keep supporting their CASE tools.

By Agile proponents. While they didn't reject UML outright, they criticized its rigidity, time consumption, and overhead of creating detailed UML diagrams that became outdated as reqrs evolved, and started looking for something better.

UML's semantic backplane

Entirety of UML semantic backplane is 'forest of rooted trees (FORT)' covering all its 308 Model Elements (ME). FORT requires that every one of those Model elements is either 'leaf' or 'root with at least one leaf'.



Using Liskov Substitution Principle, every ME in FORT assumes meaning of its root (parent), and adds to that its own unique meaning. E.g., *NamedElement* is ME with name or type; *Namespace* is Named Element named or typed container of compatible elements.

To appreciate magnitude of Generalizing 308 elements, know that it took 16 slides to represent entire suite of trees in UML's FORT.

Notational self-consistency

UML follows strict regime of assigning shape and outline of ME according to type categorization, namely structure and behaviour.

Structure is defined as collection of ME for which:

- Composition defines uniqueness (of structure instance or type).
- Absence of any part disrupts integrity and operability of whole.
- Begin and end are definitely known and may even be definitive.

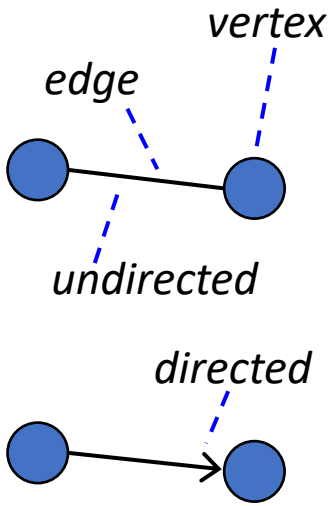
Behaviour is defined as collection of ME for which:

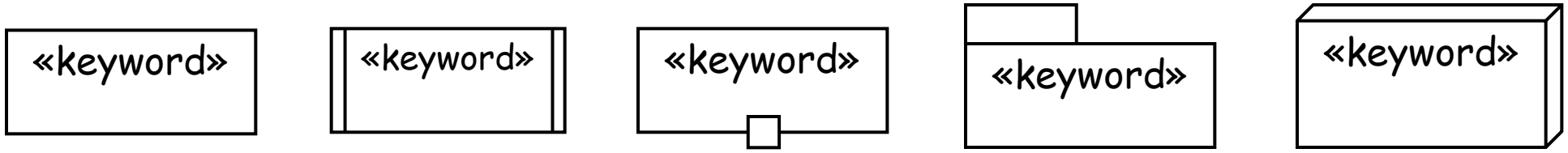
- Repetition and/or duration defines uniqueness (of behaviour instance or type).
- Compositional integrity is not definitive.
- Begin and end may be unknown or unknowable.

Let's study briefly distinguishing aspects of structure and behaviour notation in two slides next slides for Vertices and Edges notations of structure and behaviour MEs.

Notational self-consistency (cont) for Vertices

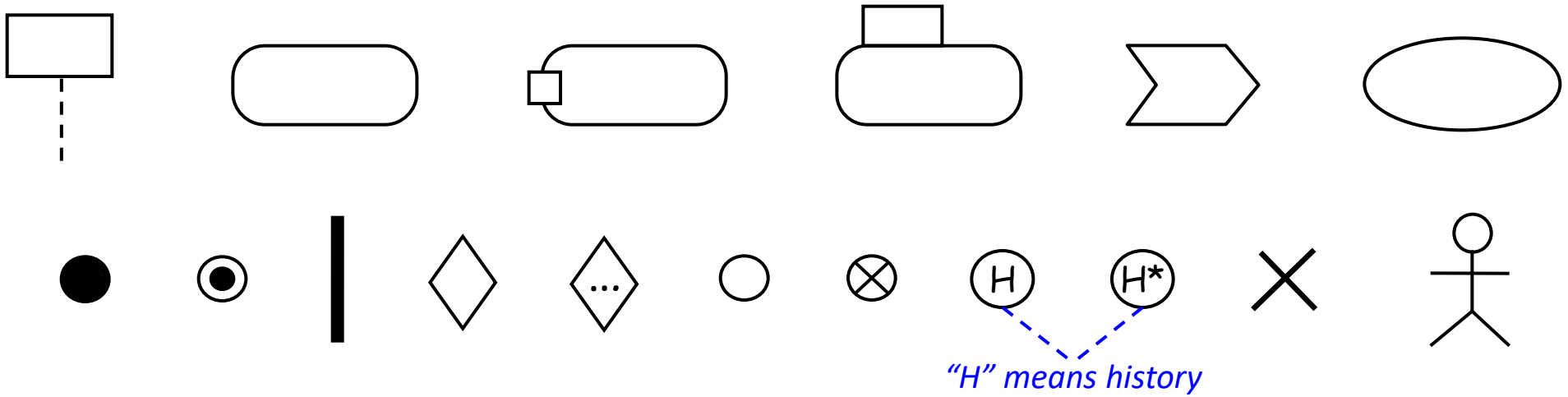
Notation follows Graph Theory.





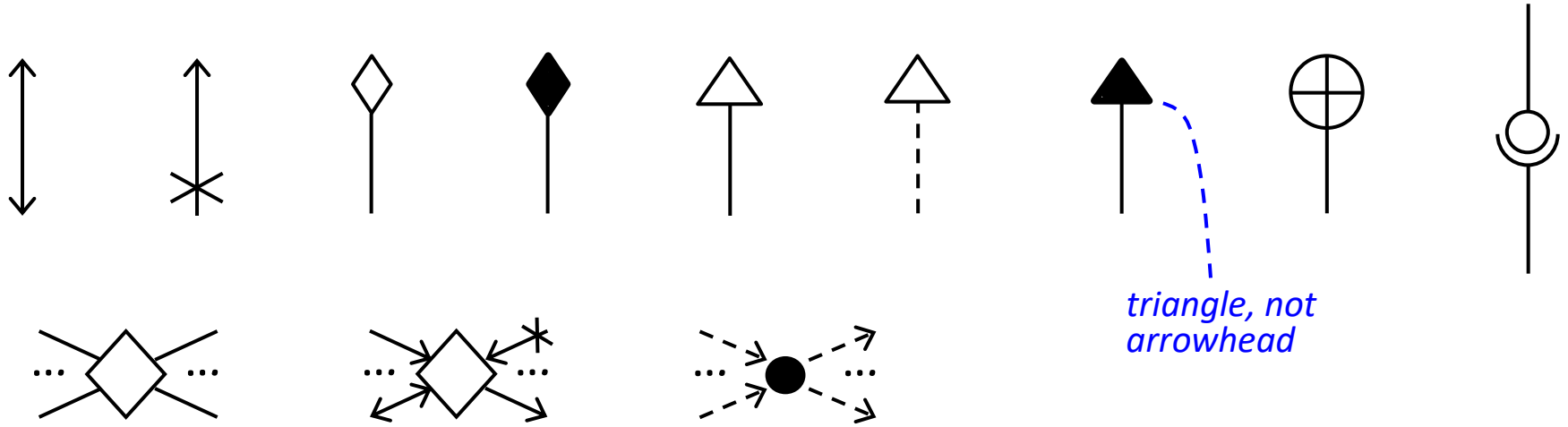
For Elements only in **structure** diagrams.

For Elements only in **behavior** diagrams.



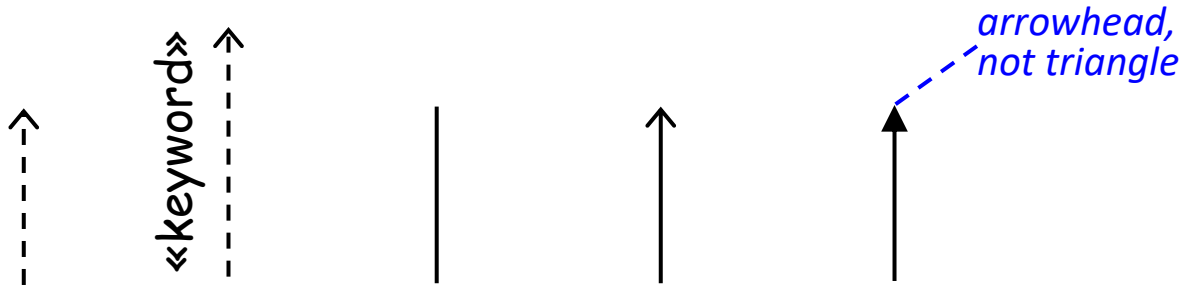
"H" means history

Notational self-consistency (cont) for Edges



For Elements only in **structure** diagrams.

For Elements only in **behavior** diagrams.



Reasons for UML's decline in usage

Study published in 2025 analyzed ~13,000 GitHub repositories over two decades and found that only 4.2% contained actual UML diagram files. Microsoft Visual Studio (major IDE) maintained native support for UML until 2016, when it was forever removed due to low client utilization. Decline of usage can be attributed to several reasons:

- **Complexity and learning curve**. UML is extensive and complex; learning to use all 14 diagram types correctly requires significant effort and modeling expertise.
- **Over-modeling**. Word to only model truly important and difficult-to-explain parts of new sw, was overshadowed by siren call to model everything. This led to excessive document-ation that provided little value; naturally being source of frustration to developers who had been obliged to produce documents for documentation's sake.
- **Tool limitations**. Early CASE tools often failed to deliver on promise of seamless code generation and faced usability issues, making modeling process an inefficient extra step.
- **Rise of Agile which works on JIT not BDUF**. Success of Agile methodologies demon-strated that comprehensive upfront modeling was not necessary for successful project delivery, especially in fast-moving environments where reqrs changed frequently.

UML's current status with sw developers

Today, UML's status is less prominent, but far from extinct. Generally, it is used selectively & pragmatically.

- **Niche and critical systems**. It remains prevalent in specific fields, such as embedded systems, safety-critical software, and enterprise architecture, where formal documentation is necessity by regulation or engineering best practice.
- **Selective communication tool**. Developers often use specific UML diagrams for specific needs: Class for analysis, prototyping & maintenance, UseCase for reqrs & testing, Activity for design & realizing priority, Sequence for communications analysis & prototyping, and Deployment for platform design and release fielding.
- **Educational contexts**. UML is still standard teaching tool in university computer science programs to help students reason about software design and testing.
- **Informal dominance**. Most modern UML usage is actually informal, consisting of hand-drawn diagrams on whiteboards that borrow UML notation to communicate complex ideas quickly within team without strictly adhering to its formal syntax.
- **Text-to-UML modeling**. There is noted resurgence of UML usage through advent of human-readable, text-to-UML-diagram tools like PlantUML and Mermaid.

Active learning

One slide

These are instructions for this session

Your team will analyse and compare performance of three text-to-UML-diagram tools of your choice: don't discount genAI in your selection. Hopefully, you remembered to bring your test sample text.

Each team should break into separate subteams to carry out analysis on three tools in parallel. Suggest ten mins for analysis and recording your findings on usability, goodness of results, and anything else that applies. For comparison to be credible, all subteams should use same test sample.

Once analysis is completed, subteams reunite to form back team. Suggest five mins to produce very brief slide presentation of results of analysis.

Teams have total of fifteen mins to finish task. Once time is up, teams be prepared to share their results if called by teacher. Feedback is available from teacher and peers. Post team's analysis 'Note' into LMS.

Any questions? Wait for teacher's signal to begin preparation.

Epilogue

Two slides

In late 1980s to mid-1990s, SWE was fragmented landscape where dozens of competing object-oriented notations created Tower of Babel for developers. UML concept drafted in 1996, was heralded as rigorous, heavyweight semantic framework that offered unprecedented clarity for complex sw design. However, seismic shift toward lightweight methodologies brought by 2001 Agile Manifesto directly challenged BDUF approaches. Exhaustive maintenance in sw dev projs of massive inheritance trees and formal diagrams were seen as bottlenecks rather than benefits, leading to sharp decline in formal UML adoption. Yet, recently UML has claimed new role as vital shorthand for communication via "sketch-based" visualizing of high-level interactions that code alone cannot easily convey.

"I'm not dead yet!"

My final words

Contact me for …

- regular stuff via my work email h51581@singaporetech.edu.sg.
- more sensitive stuff, by Whatsapp 97761716 or my personal email johnmilton2000@gmail.com.

Tomorrow's lab is 1st on subject of Innovative reqrs capture. Hopefully, your Validating was successful, and you are ready to study possible successors to RE. In preparation, ensure you are familiar with your …

- User stories.
- PD; yes, turn what you did earlier into a blank slate, and start a-fresh.
- Ques concerning how to apply UseCase2.0 + effective writing.

Keep on working through Glossary, four terms per week, and Google.

Teachers take certain concepts being understood for granted. If you are falling behind, see me or TA Ernest for consultation.

Glossary & Google readings for next week

- ???

Any final thoughts?

If not …

End of LS#12
12. UML: Giant No More – Less Relevant w/Lightweight
by Prof Kevin

ICT2113 Software Reqrs: Now & Near Future